REVERSIBLE LOGIC FRAMEWORK FOR PERFORMANCE-OPTIMIZED RADIX-2^n MDC FFT DESIGN

Ph.D. Public Viva-Voce on 30th April, 2026

Full-time Research Scholar

Anjana. A

Department of Electronics Engineering,
Pondicherry University

Research Guide

Dr. R. Nakkeeran,
Professor

Department of Electronics Engineering,
Pondicherry University

Outline

Objective

Introduction

Literature Review

Module 1 : Design of Efficient Reversible Logic Circuits for Parallel Bit Reversal

Module 2 : Unified Reversible Logic Gate for Half Add/Subtract and Swap Operations

Module 3 : Design and Analysis of RMFAS CORDIC II Rotator

Module 4 : A Novel One Step Trivial Rotator Based on Reversible Logic Gate

Module 5: Design of efficient parallel memoryless shuffling (MDC) circuit

Results

Conclusion

Future Work

Publications

References

Objective

- To design and implement a performance-optimized radix-2^n Multi-Path Delay Commutator (MDC) Fast Fourier Transform (FFT) architecture for Digital Signal Processing (DSP), focusing on minimizing latency and area overhead while achieving low-power efficiency on FPGA platforms.

Introduction

- The Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT) are key techniques in Digital Signal Processing (DSP), which converts the discrete input signal from the time domain to it's corresponding frequency domain.

- The DFT has the computation complexity of $O(N^2)$, therefore it does not meet the demands of the modern computation systems.

- FFT is an efficient algorithm to compute DFT with a reduced computation computation complexity of $O(N \log N)$.

Discrete Fourier Transform

The DFT can be expressed mathematically as shown in Eq. 1:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}, \quad k = 0, 1, \ldots, N-1 \tag{1}$$

where:

- $x[n]$ - Input time-domain sequence
- N - Total number of points or n-samples
- $X[k]$ - Frequency-domain output
- $W_N = e^{-j\frac{2\pi}{N}}$ - Twiddle factor (complex rotation)

Fast Fourier Transform

The FFT is an efficient way to compute the traditional DFT using divide-and-conquer method and is shown in Eq.2:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n] \cdot W_{N/2}^{kn} + \sum_{n=0}^{N/2-1} x[2n+1] \cdot W_{N/2}^{k(n+1)}, \quad k = 0, 1, \ldots, N-1 \quad (2)$$

where:

- $X[k]$ - Frequency-domain output
- $x[2n]$ - Even-indexed input samples
- $x[2n+1]$ - Odd-indexed input samples
- $W_N = e^{-j\frac{2\pi}{N}}$ - Twiddle factor

16-Point Radix-2 FFT Flow-Graph

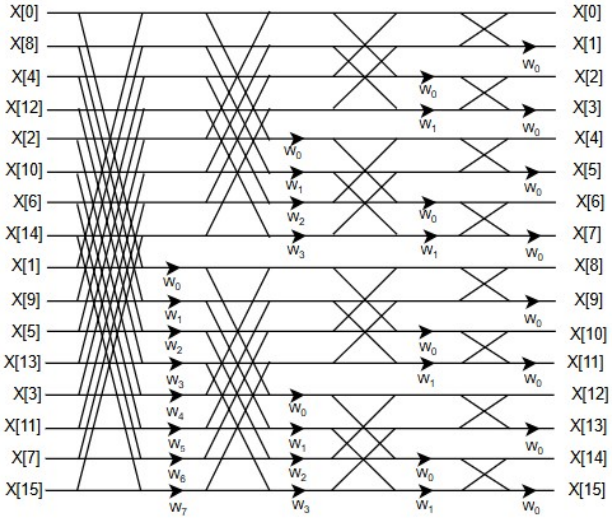


Figure. 1: 16-point radix-2 FFT flow-graph

Block Diagram of DIT FFT

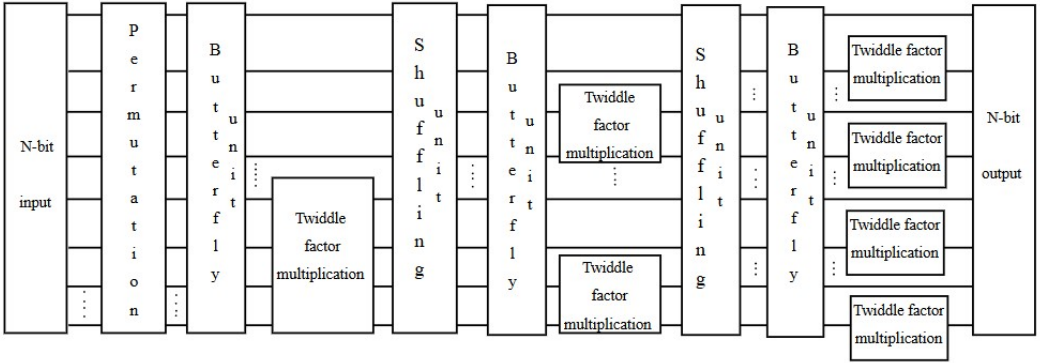


Figure. 2: Block diagram of DIT FFT

Applications of FFT

Wireless communication

Global Positioning System (GPS)

Audio or video compression

RADAR

Electrocardiogram (ECG)

Digital Signal Processing (DSP)

Literature Review

M. Garrido, "**A Survey on Pipelined FFT Hardware Architectures**," *Journal of Signal Processing Systems*, vol. 94, pp. 1345–1364, 2022.

Table 1: **Observation from [1]:** Comparison of FFT Architectures

Architecture	**Memory**	**Adders**	**Rotators**	**Latency**	**Throughput**
SDF (Serial)	Very low	Higher	Moderate	Low	1/sample
SDC (Serial)	Higher	Low	Low	Higher	1/sample
MDF (Parallel)	Low	Moderate	Moderate	Low	Multi-sample
MDC (Parallel)	Low	Low	Low	Low	Multi-sample

Inference from [1]

MDC is indeed the most hardware-efficient parallel architecture with low latency and high throughput.

Note: The quantitative analysis varies depending on the applications the FFT architecture is applied.

Continued...

k. Elango and K. Muniandi, "**A Novel Digital Logic for Bit Reversal and Address Generations in FFT Computations**," *Wireless Personal Communications*, vol. 128, pp. 1827–1838, 2023.

Inference from [18]:

It has a swap logic and 2's complement logic, which makes bit- reversal and address generation complex at hardware level.

This logic require extra swap and control logic for managing input sequencing, which could limit scalability for larger FFT sizes.

Continued...

S. Inguva and Seventiline, "**Implementation of FPGA Design of FFT Architecture Based on CORDIC Algorithm**," *International Journal of Electronics*, vol. 108, no. 11, pp. 1914–1939, 2021.

Inference form [26]

This architecture uses reversible logic for combinational circuits and conventional logic for sequential circuits, reducing power consumption, area, and delay in CORDIC II.

The hybrid implementation still requires conventional logic, so the reduction in power and complexity is limited.

Continued...

H. Zhang, M. Chen, S. Wan, Z. Jiao, S. Hu, and P. Song, "**An Arbitrary-Phase-Varying Digital Chirp Signal Generator Based on Improved CORDIC Algorithm and Trajectory Approximator**," *IEEE Access*, vol. 12, pp. 135362–135371, 2024.

Inference from [28]:

The traditional trivial rotator relies on multiplexers and negators to execute a predefined set of rotation angles.

This architecture increases area, delay, and power, thereby reducing the efficiency of rotation on FPGA.

Continued...

Z. Kaya, M. Garrido, and J. Takala, "**Memory-Based FFT Architecture With Optimized Number of Multiplexers and Memory Usage**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 8, pp. 2910–2914, Aug. 2023.

Inference from [3]:

The design uses only the hardware multiplexer per branch and unified memory addressing, enabling merging of banks for a compact in-place memory architecture.

The design has lower speed and higher area occupancy, while it reduces DSP and BRAM usage, it requires more logic slices.

Continued...

S. Wang and J. Chen, "**A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 2, pp. 832-836, Feb. 2024.

Inference from [7]:

Input grouping and partial sum sharing together reduce multipliers and adders, saving area and power in small/medium FFT sizes.

Extra control logic for grouping and sharing increases complexity; at larger FFT sizes it reduces scalability and may limit speed despite area and power savings.

Design of Efficient Reversible Logic Circuits for Parallel Bit Reversal

Block Diagram of DIT FFT

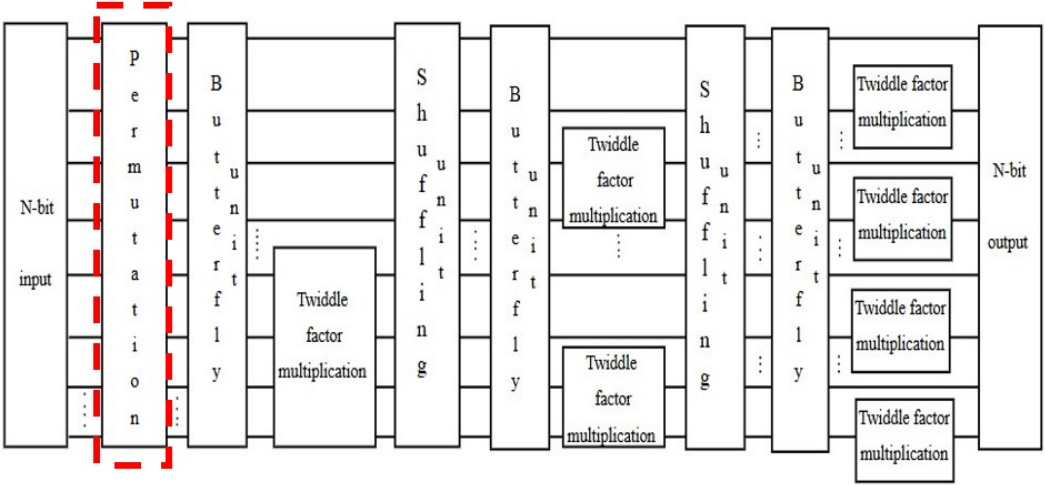


Figure. 3: Block diagram of DIT FFT

Introduction

- The permutation block facilitates the reordering of the N-**input data streams**, transforming them **from natural order into index bit-reversed order**, and conversely enabling the inverse mapping.
- The permutation is typically realized using two fundamental architectures: **Delay-based architectures**: Exploit controlled timing offsets to systematically reorder the input sequences. **Memory-based architectures**: Employ storage elements to temporarily buffer data streams for the sequence reorganization.

Limitations in Existing Methods:

- Memory-based architectures consume more memory and occupy more area in FPGA devices.
- Delay-based architectures suffer from increased synchronization overhead and design complexity when scaled to larger input sizes.

Merits of Proposed Methods:

- Reduces the number of input sequences for bit reversal operation and synchronization overhead.
- Minimizes multiplexer usage through hypercube-based simplified XOR based Palindrome Check Inverse (XPCI) circuits.

Continued...

- The permutation architecture proposed in this paper is specifically designed for parallel input indices and adopts pipelining method, which explores the hypercube technique for reducing the bit-reversal processing of index sequences.
- Firstly, the N-point parallel data stream is bifurcated into N/2-blocks based on hypercube layering technique as: α-layer and β-layer. Further, for one of the layers, the palindromic sequences are bypassed and the non-palindromic sequences are bit reversed.
- Secondly, the outcome of the bit reversal circuit is rotated to 180^0 and is negated to generate the bit-reversal of β-layer. The proposed delay-based bit reversal circuit can be implemented using XOR, NOT gates and multiplexers with optimized gate count and frequency.
- The power consumption of is optimized in XPCI circuits by following the reversible logic footprints.

Proposed bit reversal permutation using hypercube

- Hypercubes are n-dimensional cubes with each vertex representing a value of binary index. The binary indices are arranged on hypercube with only one-bit difference between the adjacent vertices and is interpreted before permutation as (000), (001), (010), (011), (100), (101), (110), (111).
- Bit reversal permutation is carried out for $N = 2^n$ binary sequence with its data index, I is represented as $I = b_{n-1}, b_{n-2}, \ldots, b_1, b_0, \quad b_i \in \{0, 1\}$ and the permutation sequence is denoted by π_i. Its general equation is given by Eq. 3

$$\pi(b_{n-1}, b_{n-2}, \ldots, b_1, b_0) = b_{\pi(n-1)}, b_{\pi(n-2)}, \ldots, b_{\pi(1)}, b_{\pi(0)}. \tag{3}$$

- The index before permutation is denoted as a function of π, the index after permutation is denoted as a function of $\pi^{'}$. The order of input index is given by ϱ_0 and the output index order is given by ϱ_1 as in Eq. 4.

$$\pi(\varrho_0) \quad \rightarrow \quad \pi'(\varrho_1) \tag{4}$$

Hypercube Layering Technique

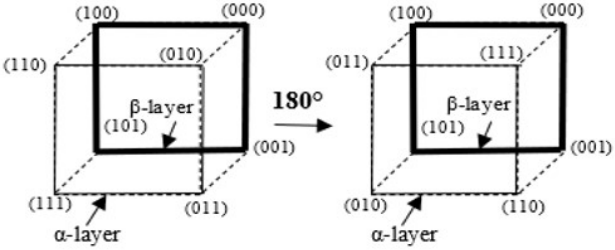


Figure. 4: Illustration of $\alpha - layer$ rotation on hypercube

$$\pi(010),\ \pi(011),\ \pi(111),\ \pi(110) \xrightarrow{(90^\circ)} \pi(011),\ \pi(111),\ \pi(110),\ \pi(010) \tag{5}$$

$$\pi(011),\ \pi(111),\ \pi(110),\ \pi(010) \xrightarrow{(180^\circ)} \pi(111),\ \pi(110),\ \pi(010),\ \pi(011) \tag{6}$$

$$\pi(111),\ \pi(110),\ \pi(010),\ \pi(011) \xrightarrow{(\text{negator})} \pi(000),\ \pi(001),\ \pi(101),\ \pi(100) \tag{7}$$

Mathematical Interpretation

- The hypercube technique-based bit reversal permutation, some indices retain their binary representation before and after bit reversal, which is palindromic in nature.
- Bypassing those sequences reduces the hardware circuit complexity. The Eq. 8 and Eq. 9 mathematically checks the identical indices in the odd and even values of n.

Condition: if n is odd

$$(\varrho_{oi} - a) = \pi(\varrho_{oi} + b), \quad \begin{cases} i = 1, 2, \ldots, (n-2) \\ a = 1, 2, \ldots, \frac{(n-1)}{2} - 1 \\ \varrho_o = n \\ b = 1, 2, \ldots, \frac{(n+1)}{2} - 1 \end{cases} \tag{8}$$

Continued...

Condition: if n is even

$$(\varrho_{ei} - a) = \pi(\varrho_{ei} + b), \quad \begin{cases} i = 1, 2, \ldots, \frac{n}{2} \\ a = 1, 2, \ldots, \frac{(n)}{2} \\ \varrho_o = n \\ b = 1, 2, \ldots, \frac{(n)}{2} \end{cases} \tag{9}$$

Continued...

Table 2: Calculation of identical index for N-bit sequences

n	**N=**2^n	η**(I.I)**
1	2	NA
2	4	N/2
3	8	N/2
4	16	N/4
5	32	N/4
6	64	N/8
7	128	N/8
8	256	N/16
9	512	N/16
10	1024	N/32

Continued...

If the index values for n are identical, no inversion is required. Otherwise, the bits are negated ($0 \rightarrow 1$ or $1 \rightarrow 0$) for each intermediate condition, as shown in Eq. 10 and Eq. 11.

Condition: if n is odd

$$b_{\left(\frac{(n+1)}{2}-a\right)} = b_{\left(\frac{(n+1)}{2}+b\right)}, \quad b_{\left(\frac{(n+1)}{2}-a\right)} \neq b_{\left(\frac{(n+1)}{2}+b\right)} \begin{cases} i = 1, 2, \ldots, \frac{n-1}{2} - 1 \\ a = 1, 2, \ldots, \frac{(n+1)}{2} - 1 \end{cases} \tag{10}$$

Condition: if n is even

$$b_{\left(\frac{n}{2}-a\right)} = b_{\left(\frac{n}{2}+b\right)}, \quad b_{\left(\frac{n}{2}-a\right)} \neq b_{\left(\frac{n}{2}+b\right)} \begin{cases} i = 1, 2, \ldots, \frac{n}{2} \\ a = 1, 2, \ldots, \frac{(n)}{2} \end{cases} \tag{11}$$

where n=1,2,3,....

Timing Diagram of Proposed Delay-Based Bit Reversal Permutation

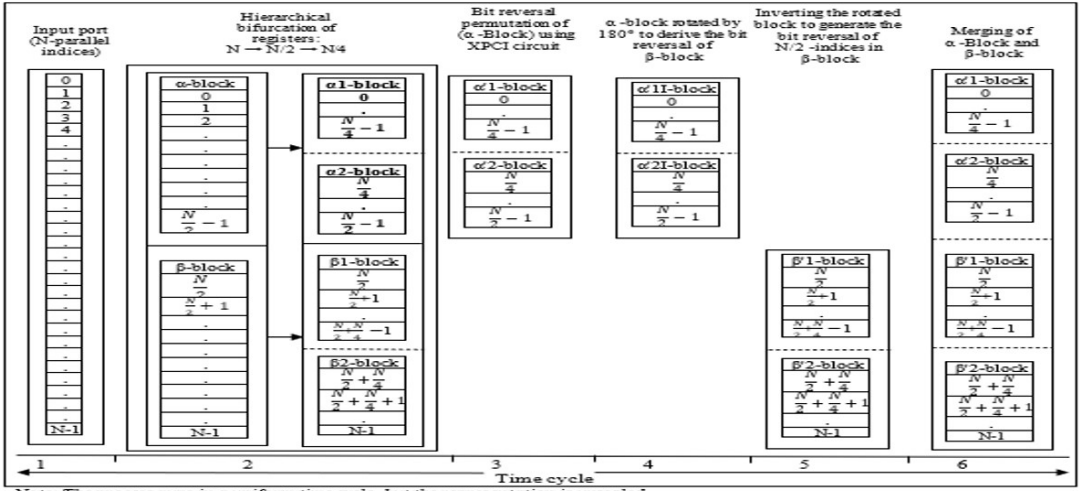
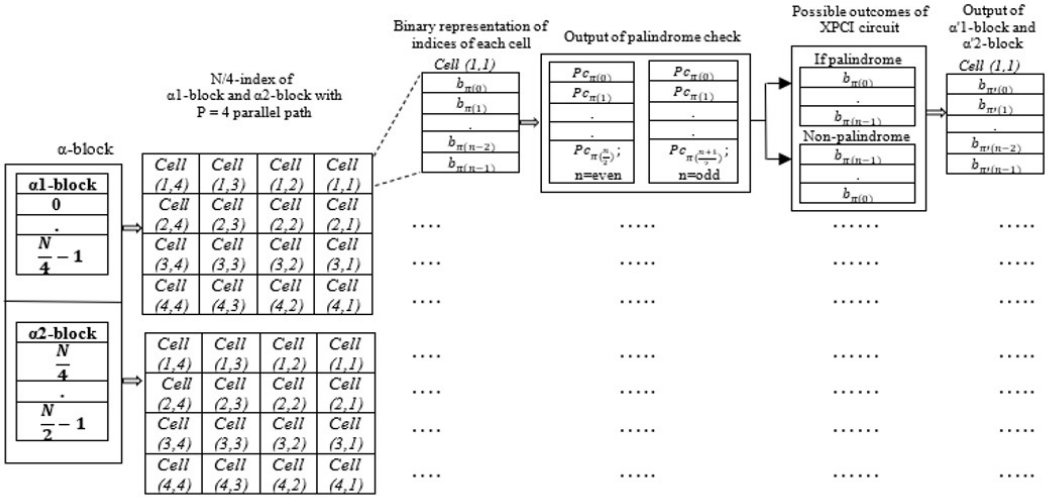


Figure. 5: Timing diagram of the proposed 4-parallel path N-sequence bit reversal permutation

Proposed XPCI algorithm 4-parallel path N/4-sequence



Note: The ellipsis in the figure represent that the above processes are iterated in parallel to each cell of α-block.

Figure. 6: Illustration of the proposed bit reversal XPCI algorithm for 4-parallel path N/4-sequence

Proposed XPCI bit reversal circuit

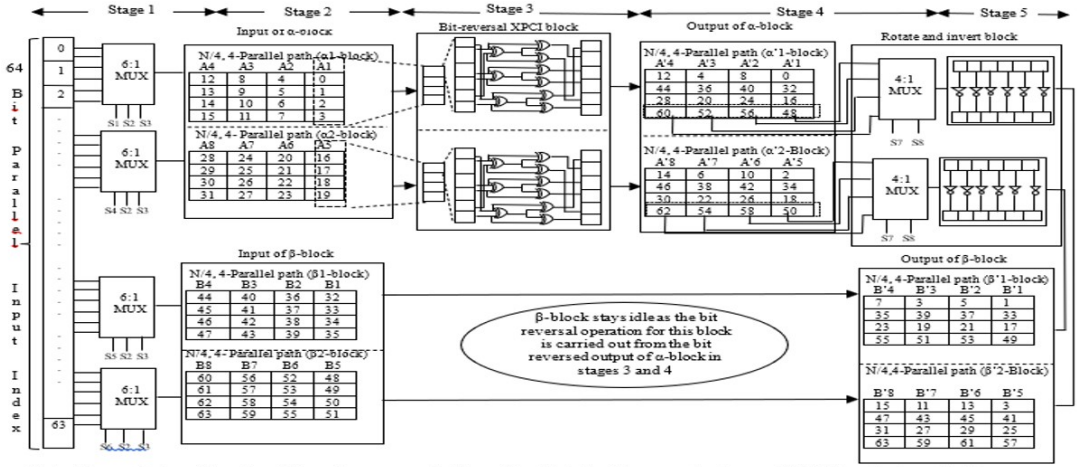


Figure. 7: Illustration of the proposed 4-parallel path 64-bit XPCI bit reversal circuit

Rectangular pattern method for XOR gate count

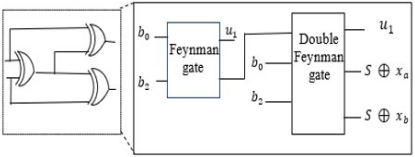


Figure. 8: Implementation of XPCI circuit in reversible logic

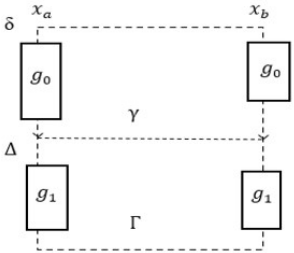


Figure. 9: Rectangular pattern for binary input quartets

Note: This method is adopted here only for the purpose of finding the empirical XOR gate count

Continued...

- The rectangles are constructed using the probabilities of quartets γ and δ for g_0 and g_1, and their corresponding output differences Δ and Γ.
- Each rectangle represents a set of input-output pairs with the input differences permutation sequences. The quantitative rectangular quartets, $\eta(R.Q)$, and the total $\eta(R.Q)$ are given by Eq. 12 and Eq. 13, respectively.

$$\eta(R.Q) = \begin{cases} \frac{n}{2}, & \text{if } n \text{ is even} \\ \frac{n+1}{2}, & \text{if } n \text{ is odd} \end{cases} \tag{12}$$

$$\sum \eta(R.Q) = \begin{cases} N(\frac{n}{2}), & \text{if } n \text{ is even} \\ N(\frac{n+1}{2}), & \text{if } n \text{ is odd} \end{cases} \tag{13}$$

Results

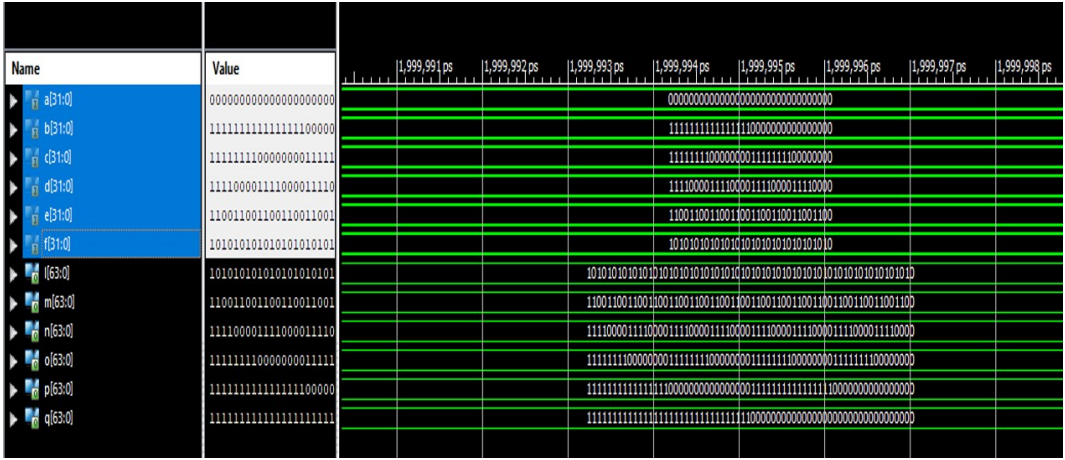


Figure. 10: Simulation result of proposed XPCI bit reversal architecture

Continued...

Table 3: Comparison of architectural design for parallel bit reversal

Work	Approach	Memory size	Latency (s)	MUX count	Cases
Garrido [9]	Memory	$\frac{N}{P}$	-	-	-
Yang [10]	Memory	$\frac{(9/8)N+192}{P}$	-	$2P^2 - P$	-
Chen [11]	Memory	N	$\frac{N}{P} - \frac{\sqrt{N/2}}{P} + 1$	-	n odd
Li [12]	Delay	$N - 2\sqrt{N} + P$	$\frac{N}{P} - \frac{2\sqrt{N}}{P} + 1$	$4P^2 + 4P + 2P[\lfloor \log_2 n \rfloor /2]$	n even
		$N - \sqrt{2N} - N/2 + P$	$\frac{N}{P} - \frac{\sqrt{2N}}{P} - \frac{N}{2P} + 1$		n odd
Garrido [13]	Memory	$N - 2\sqrt{N} + P$	$\frac{N}{P} - \frac{2\sqrt{N}}{P} + 1$	$P \log_2 P$	n even
Gyanendra [14]	Delay	$N - 2\sqrt{N} + P$	$\frac{N}{P} - \frac{2\sqrt{N}}{P} + 1$	$\frac{P}{2}(P+1) + 2P[\lfloor \log_2 n \rfloor /2]$	n even
Konguvel [18]	Memory	N	N	-	-
Proposed	Delay	$N - \sqrt{2N} + P$	$\frac{N}{P} - \frac{\sqrt{2N}}{2P} + 1$	$N/4$	n even
		$\frac{N}{2} + P + \frac{(1-\sqrt{2})N}{2P} - \sqrt{2N}$	$\frac{P-(2-3\sqrt{2})N}{2P} + 1$		n odd

Continued...

Table 4: Comparison of different works for $N = 64$, $P = 4$, $n = 6$.

Work	**Approach**	**Memory size**	**Latency (s)**	**Multiplexer count**	**Case**
Garrido [9]	Memory	48	–	–	–
Yang [10]	Memory	264	–	–	–
Chen [11]	Memory	64	9	28	n even
		64	16	28	n odd
Li [12]	Delay	52	13	90	n even
		51	13	90	n odd
Garrido [13]	Memory	64	16	8	–
Gyanendra [14]	Delay	52	13	10	n even
		25	6	10	n odd
Proposed work	**Delay**	**25**	**8**	**16**	**n even**
		21	**10**	**16**	**n odd**

Continued...

Table 5: Comparison of 64-point bit-reversal

	Garrido, et al. [13]	**Gyanendra, et al. [14]**	**Proposed work**	**% Improvement wrt [14]**
Gate count	2755	1809	496	72.6
Frequency (MHz)	558	826	1326.2	60.6
Power consumption (mW)	6.6	5.92	5.22	11.8

Summary

- The adoption of the hypercube architecture, specifically through the α-layer and β-layer structure, reduces the gate count and the number of sequences undergoing bit-reversal permutation.
- The use of reversible logic gates in the XPCI circuit results in reduced power consumption.
- The proposed XPCI bit reversal method shows substantial improvements:
  - Reduction in gate count and number of sequences by approximately 72.58%.
  - Reduction in power consumption by approximately 11.82%.
  - Increase in operating frequency by approximately 60.55% compared to the existing method.

Unified Reversible Logic Gate for Half Add/Subtract and Swap Operations

Block Diagram of DIT FFT

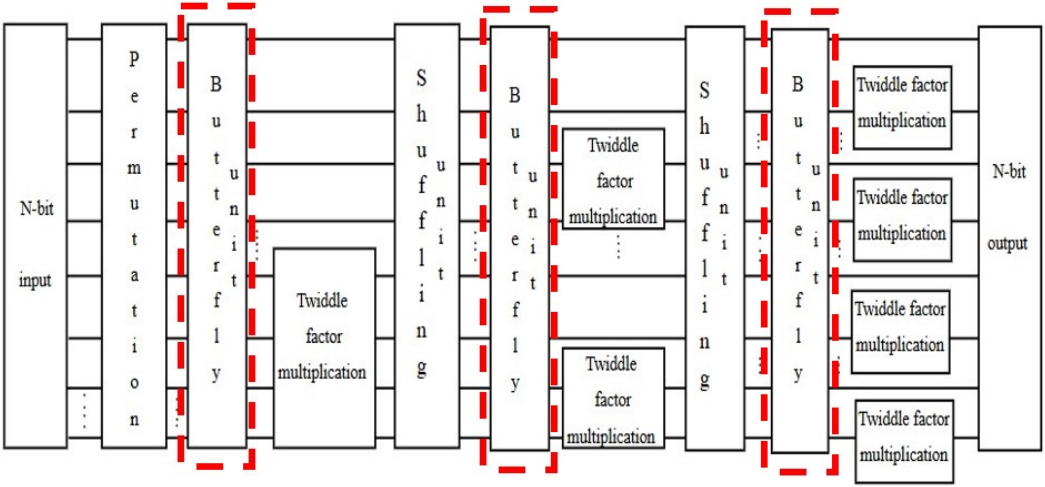


Figure. 11: Block Diagram of DIT FFT

Introduction

Arithmetic and Swap operations are the most frequent operations in FFT, especially in Radix-2 butterfly units. Use of reversible logic gates for arithmetic operations prevents the data loss due to entropy.

Limitations in existing gates:

- Existing gates, on attempting to reduce the quantum cost and garbage values results in trade off between the quantum cost and garbage values.

Merits of proposed gate:

- The proposed reversible Half Add Subtract Swap HAS^2 gate performs arithmetic operations with reduction in both quantum cost $= 4$ and garbage value $= 0$.
- Also, performs swap operations along with other Boolean functions namely copy, AND, XOR, XNOR, NOT under certain input conditions.

Preliminaries of Reversible Logic Gate

- Definition 1: A reversible logic gate is a bijective gate.
- Definition 2: Constant inputs such 0's and 1's are fed to the reversible logic for performing Boolean functions.
- Definition 3: Reversible logic gates produce outputs as functionality output and garbage output. Garbage values are used for data retrieval.

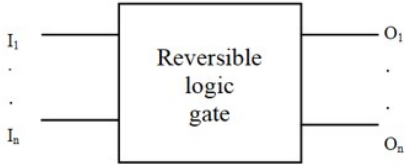


Figure. 12: *nxn*-bit reversible logic gate

Proposed Reversible HAS^2 Gate

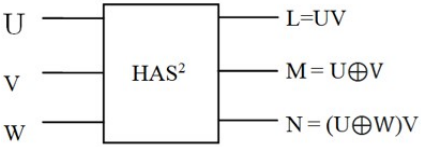


Figure. 13: 3x3 Reversible HAS^2 gate

- The reversible 3x3 HAS^2 gate, which has 3 input and three output ports.
- It has the quantum cost of 4.

Continued...

$$f_{\text{in}}\{A, B, C\} = f_{\text{out}}\{P, Q, R\} = f_{\text{out}}\{AB,\ A \oplus B,\ (A \oplus C)B\} \tag{14}$$

Figure. 14: Simulation result of HAS^2 gate

Continued...

Table 6: Input-Output Table

A	B	C	P	Q	R
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	0	1	0
1	0	1	0	1	0
1	1	0	1	0	1
1	1	1	1	0	0

HAS^2 Gate as Half Adder/Subtractor (HA/HS)

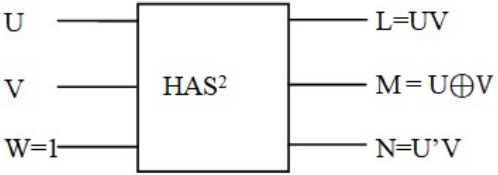


Figure. 15: Reversible HAS^2 gate as HA/HS

$$
\begin{aligned}
f_{\text{in}}\{A, B, C\} &= f_{\text{out}}\{AB,\ A \oplus B,\ (A \oplus C)B\} \\
f_{\text{in}}\{A, B, 1\} &= f_{\text{out}}\{AB,\ A \oplus B,\ (A \oplus 1)B\} \\
&= f_{\text{out}}\{AB,\ A \oplus B,\ A'B\}
\end{aligned} \tag{15}
$$

Continued...

Table 7: Condition-based Input-Output of HAS^2 Gate

Condition	A	B	C	P	Q	R	Result
C = 1	A	B	1	AB	$A \oplus B$	$A'B$	Carry, Sum/Difference, Borrow

Figure. 16: Simulation result of HAS^2 gate as HA/HS

HAS^2 Gate as Swap Gate

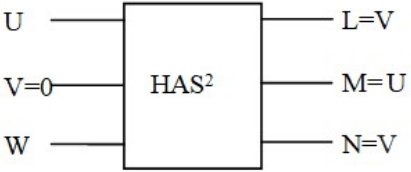


Figure. 17: Reversible HAS^2 gate as Swap gate

$$
\begin{aligned}
f_{\text{in}}\{A, B, C\} &= f_{\text{out}}\{AB,\ A \oplus B,\ (A \oplus C)B\} \\
f_{\text{in}}\{A, 0, C\} &= f_{\text{out}}\{A.0,\ A \oplus 0,\ (A \oplus C)0\} \\
&= f_{\text{out}}\{B,\ A,\ B\}
\end{aligned}
\tag{16}
$$

Continued...

Table 8: Condition-based Input-Output of HAS^2 Gate

Condition	A	B	C	P	Q	R	Result
B = 0	A	0	C	B	A	B	Copy B, Copy A, Copy B

Figure. 18: Simulation result of HAS^2 gate as Swap gate

Other Boolean Functions of HAS^2 Gate

Table 9: Other Boolean operations of the proposed HAS^2 gate

Input			Output			Resultant Function
U	**V**	**W**	**L**	**M**	**N**	
1	V	W	V	V'	VW'	Copy, NOT, G1
U	1	W	UV	U'	$U \oplus W$	AND, NOT, XOR
0	V	W	U	V	UV	Copy, AND, Copy
U	V	0	UV	$U \oplus V$	UV	AND, XOR, AND
U'	V	W	$U'V$	$U' \oplus V$	$(U \cdot W)V$	G1, XNOR, G2
U	V'	W	UV	$U \oplus V'$	$(U \cdot W)V'$	G1, XNOR, G2
U	V	W	UV	$U \oplus V$	$(U \cdot W')V$	AND, XOR, G1

Comparison of HAS^2 Gate with Existing Reversible Gates

Gate	Function	Number of inputs	Quantum cost	Garbage value
NR gate [19]	HA/HS	4	7	1
MOG gate [20]	HA/HS	4	5	2
RSG gate [21]	HA/HS	3	5	0
Universal G^{3}gate [22]	HA	3	4	1
Proposed HAS^2 gate	**HA/HS**	**3**	**4**	**0**
Fredkin gate [23]	Swap	3	5	1
NCT gate library [24]	Swap	2	3	0
Proposed HAS^2 gate	**Swap**	**3**	**4**	**1**

Summary

- The proposed unified reversible HAS^2 gate supports both arithmetic and swap operations with a low quantum cost of 4.
- It produces zero garbage for HA/HS operations and only one garbage for swap operations.
- Use of HAS^2 gate for the butterfly unit of Radix-2 FFT reduces the hardware cost, area, memory and garbage value.

Design and Analysis of RMFAS CORDIC II Rotator

Block Diagram of DIT FFT

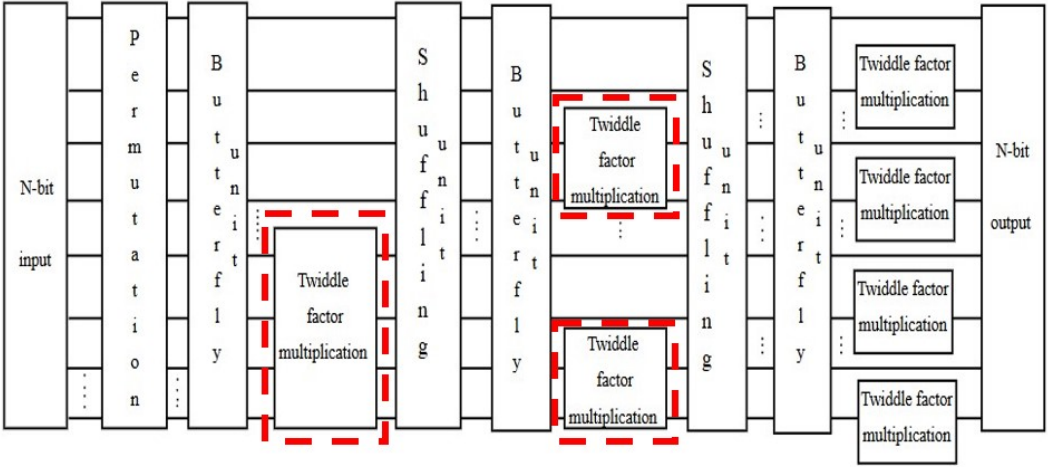


Figure. 19: Block Diagram of DIT FFT

Introduction

- CORDIC (Coordinate Rotation Digital Computer) rotator executes trigonometric functions using simple shift-add operations, enabling efficient vector conversion from rectangular to polar coordinates in FFT.
- It effectively performs the twiddle factor multiplication required in FFT computations.

Limitations:

- In existing CORDIC architecture, emphasis is in reducing the arithmetic multiplication operations and not much attention is given in reducing the adder/subtractor operations.
- This eventually results in higher hardware complexity, area, delay and power consumption.

Merits of proposed RMFAS CORDIC II rotator:

- The proposed Reversible Multiplexed Fused Add Subtract (RMFAS) CORDIC II rotator eliminates the use of adder/subtractor and replaces them with multiplexers, which reduces the hardware complexity, area and delay.
- Every sequential and combinational operations in CORDIC framework are realized using the reversible logic gates to reduce the power dissipation , compared to the existing methods.

CORDIC Algorithm

Converts a vector from rectangular to polar coordinates, enabling trigonometric computations for twiddle-factor generation in FFT. Equations 17 and 18 show the general rotation operation applied to the input vector X_{in} and Y_{in}. In vectoring mode, the algorithm determines the unknown angle θ under micro-scaling.

$$X_{\text{out}} = X_{\text{in}} \cos\theta - Y_{\text{in}} \sin\theta \tag{17}$$

$$Y_{\text{out}} = X_{\text{in}} \sin\theta + Y_{\text{in}} \cos\theta \tag{18}$$

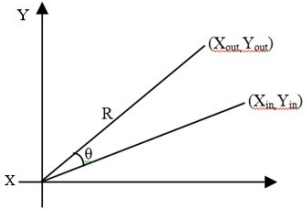


Figure. 20: Vector rotation in CORDIC

RMFAS CORDIC II Rotator Stages

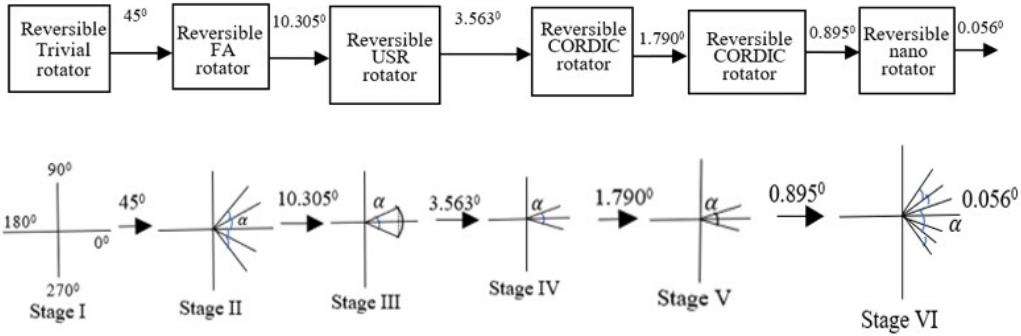


Figure. 21: Proposed reversible RMFAS CORDIC II rotator block diagram and its corresponding rotation angle

Integral Components of Each Stages

- Each and every stages of CORDIC II comprised of various components as: Adder/ Subtractor, Shift registers, Multiplexers.
- Among them the arithmetic units are the most power, area and time consuming.
- To reduce all these metrics, a technique of Reversible Muxed Fused Add-Subtract (RMFAS) operation is proposed for arithmetic units.

Existing Parallel Add-Subtract Architecture

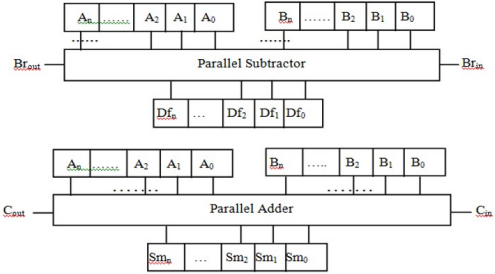


Figure. 22: Existing Add/subtract unit in CORDIC rotator

Eq. 19 shows the delay for the existing adder block

$$T_d = T_{add} + T_{2C} \tag{19}$$

Eq.20 shows the delay for the existing subtractor block

$$T_d = 2T_{add} + T_{2C} \tag{20}$$

Existing Fused Add-Subtract Architecture

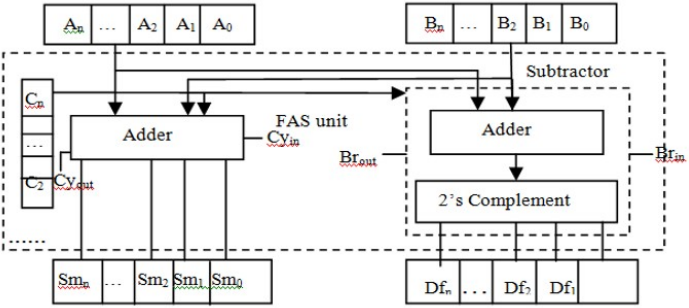


Figure. 23: Existing Fused Add-Subtract Architecture

Eq. 21 shows the delay for the existing fused add-subtract unit

$$T_d = 2T_{add} + T_{MUX} + T_{2C} \tag{21}$$

Proposed Mux'ed Fused Add-Subtract Architecture

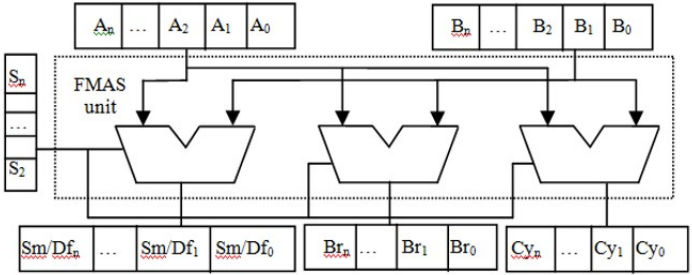


Figure. 24: Proposed Mux'ed Fused Add-Subtract (MFAS) unit

Eq. 22 shows the delay for the existing fused add-subtract unit

$$T_d = T_{MUX(add)} + T_{MUX(sub)} \tag{22}$$

Proposed MFAS Internal Architecture

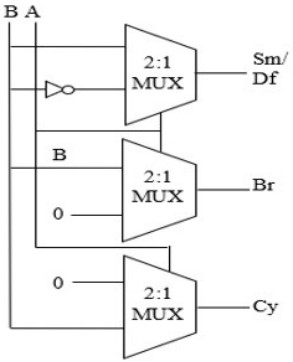


Figure. 25: Proposed Mux'ed Fused Add-Subtract (MFAS) unit

Table 10: Truth table for generating 'Sm/Df'

A	B	B'	Sm/Df
0	0	1	0
0	1	0	1
1	0	1	1
1	1	0	0

Table 11: Truth table for generating 'Br'

A	B	CI	Br
0	0	0	0
0	1	0	1
1	0	0	0
1	1	0	0

Table 12: Truth table for generating 'Cy'

A	CI	B'	Cy
0	0	0	0
0	0	1	0
1	0	0	0
1	0	1	1

Existing Reversible Integrated Add-Subtract (RIAS) Unit

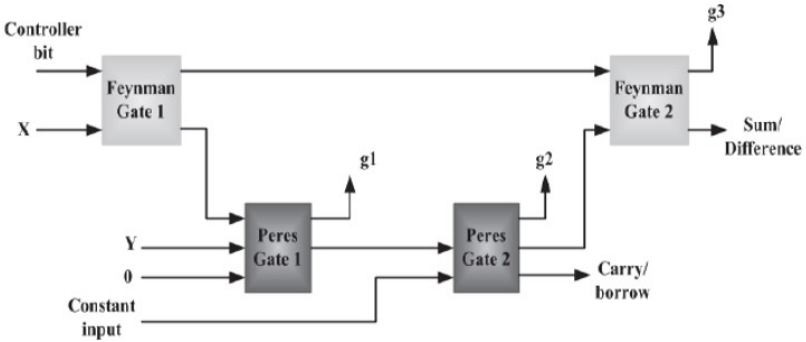


Figure. 26: Existing reversible Integrated Add-Subtract unit [25]

Proposed Reversible Mux'ed Fused Add-Subtract Architecture

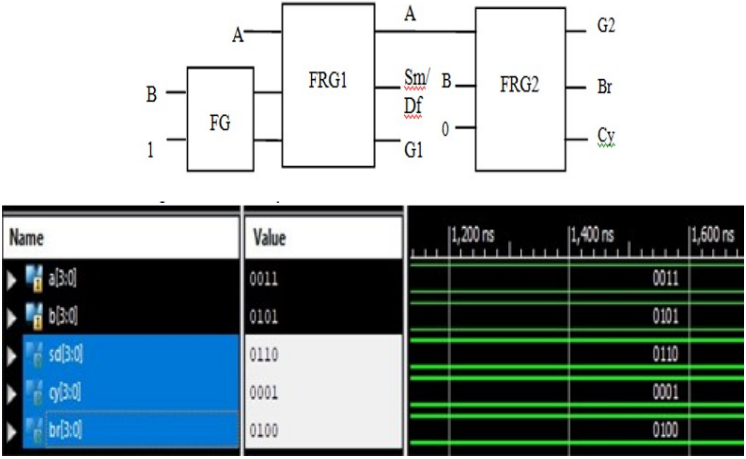


Figure. 27: Architecture and simulation of proposed RMFAS unit

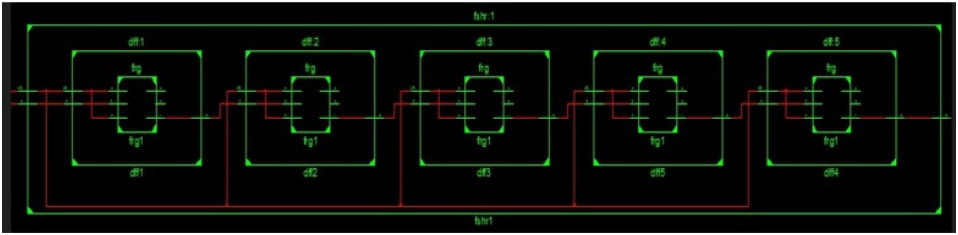
Comparison of RIAS and RMFAS unit

Table 13: Comparison of RIAS and RMFAS unit over Virtex7 FPGA device

Metrics	**RIAS unit [25]**	**RMFAS unit [Proposed]**
Number of reversible logic gate	4 (HA) + 4 (HS) $= 8 \times 4 = 32$	3 (HA and HS) $= 3 \times 4 = 12$
Garbage value	3 (HA) + 3 (HS) $= 6 \times 4 = 24$	2 (HA and HS) $= 2 \times 4 = 8$
Quantum cost	10 (HA) + 10 (HS) $= 20 \times 4 = 80$	11 (HA and HS) $= 11 \times 4 = 44$
Area (LUTs)%	-	1
Latency (ns)	-	0.799

Proposed Reversible logic gate based shift register

This paper presents a novel approach to implementing shift registers using reversible logic specifically through a D flip-flop design based on Fredkin gates



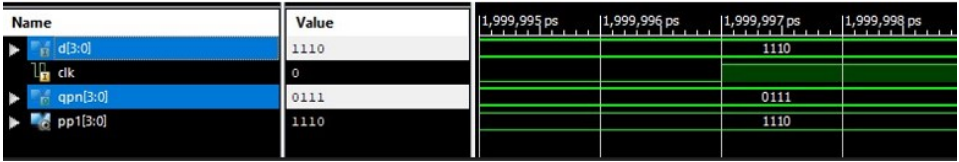


Figure. 28: RTL schematic and simulation of proposed RMFAS unit

Stage 1: Trivial Rotator

- It rotates to an angle of 0°, 90°, 180°, and 270°.
- It has two negators and four 2:1 multiplexers.

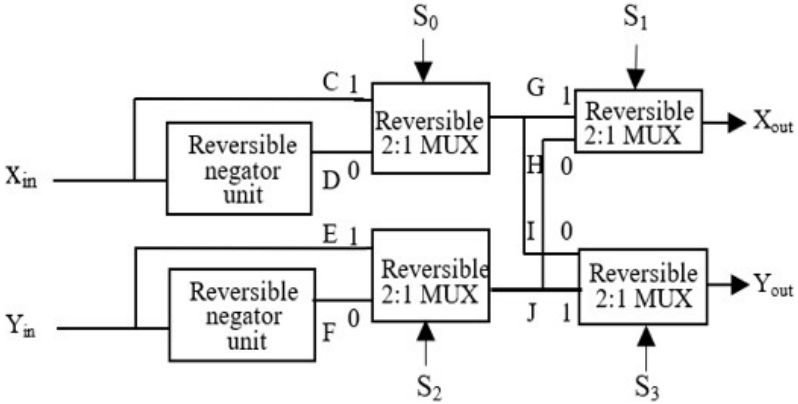


Figure. 29: Architecture of proposed trivial rotator

Stage 2: Friend Angle Rotator

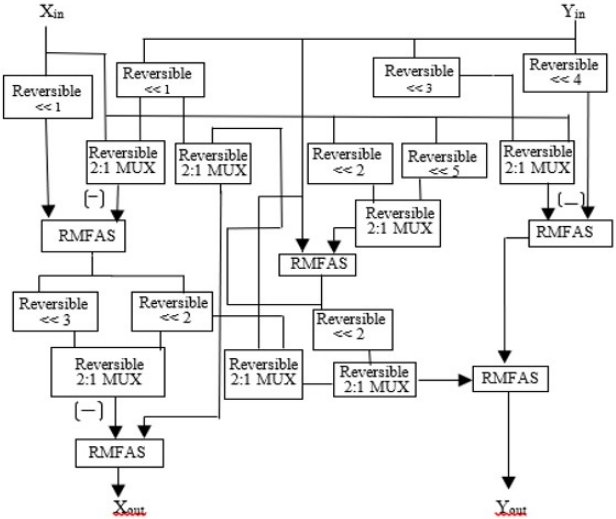


Figure. 30: Architecture of proposed friend angle rotator

Stage 3: Uniformly scaled Redundant (USR) rotator

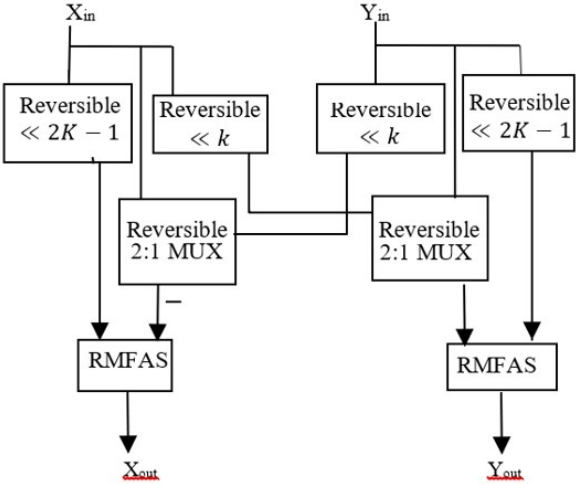


Figure. 31: Architecture of proposed USR rotator

Stage 4 and 5: CORDIC rotator

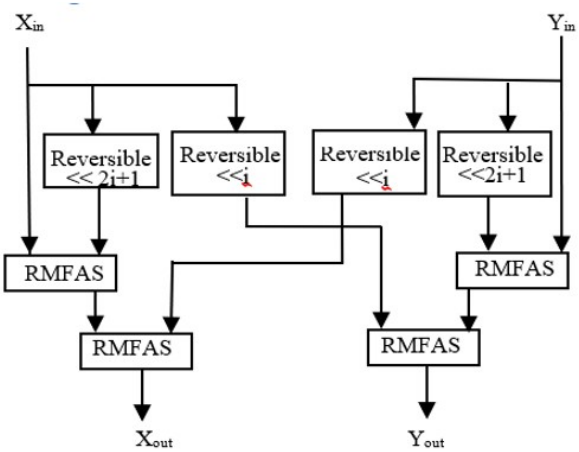


Figure. 32: Architecture of proposed CORDIC rotator

Stage 6: Nano rotator

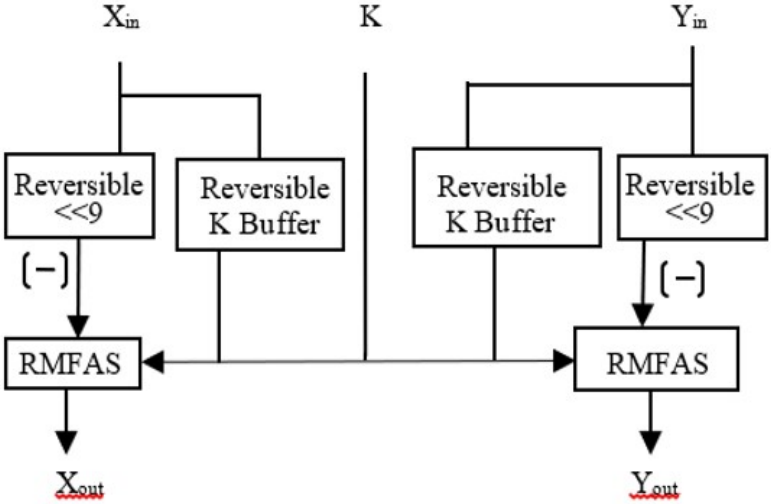


Figure. 33: Proposed reversible RMFAS Nano rotator

Scaling Architecture

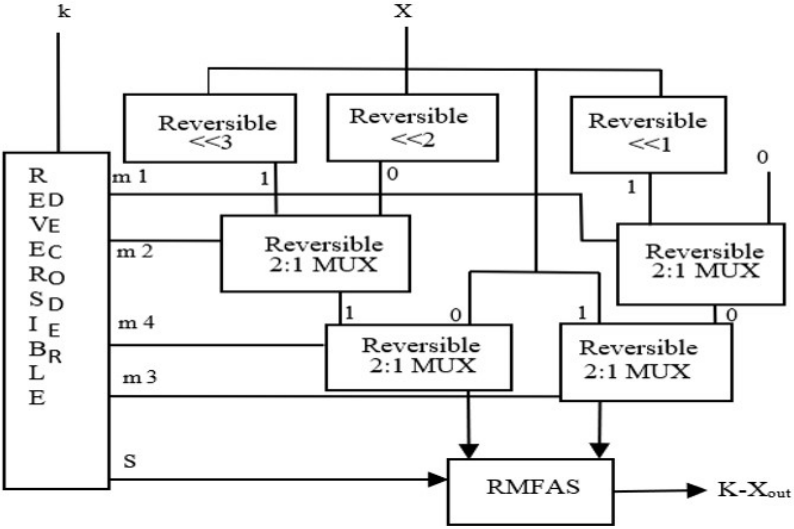


Figure. 34: Proposed reversible scaling architecture

Simulation results

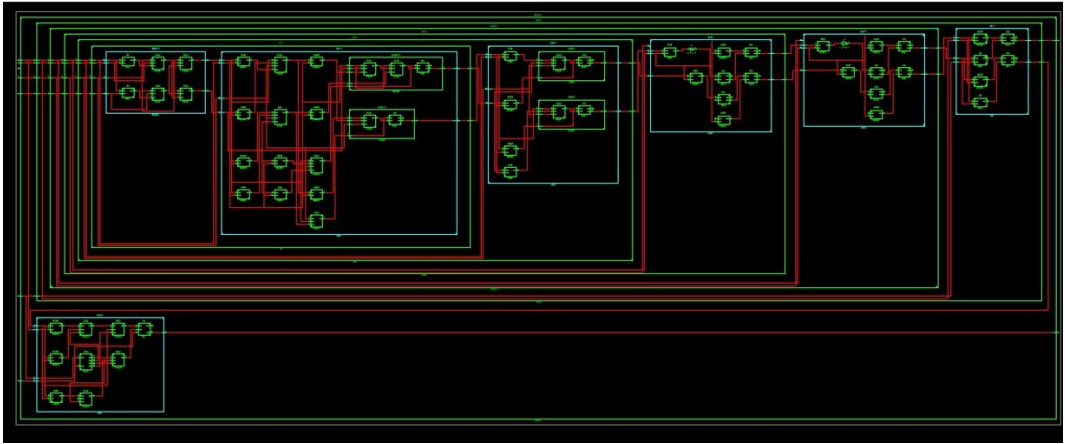


Figure. 35: RTL schematic of the proposed reversible RMFAS CORDIC II rotator

Continued...

Figure. 36: Simulation result of proposed RMFAS CORDIC II rotator

Continued...

- X and Y are the inputs, and X_o and Y_o are the outputs of the cosine and sine terms, respectively.
- Given:
  - $X = 0.1100_2 = 0.75_{10}$
  - $Y = 1011_2 = 0.6875_{10}$
- Output produced:
  - $Y_o = 0.1011_2 = 0.6875_{10}$
  - $X_o = 0.1110_2 = 0.875_{10}$
- After passing through CORDIC II rotation stages:
  - $\sin(Y = 0.75) = 0.6816 \approx Y_o$
  - $\cos(X = 0.6875) = 0.7728 \approx X_o$
- Deviation in the accuracy
  - $\sin(Y = 0.75) = 0.6875 - 0.6816 = 0.6875 - 0.6816 = 0.0059 \approx Y_d$
  - $\cos(X = 0.6875) = 0.875 - 0.7728 = 0.1022 \approx X_d$

Continue. . .

Table 14: Comparison of improvement in existing and proposed CORDIC II rotator over VIRTEX7 devices

Approaches	**No.of Add-Sub unit**	**No.of 2:1 MUX**	**Area (%)**	**Frequency (%)**
LH-CORDIC II [27]	15	31	35.82	77.11
RIAS CORDIC II [25]	15	31	31.67	76.11
RMFAS CORDIC II (proposed)	0	47	77.87	95.76

Why? MUX is Better than Adder/Subtractor in FPGA/VLSI

- MUX utilizes fewer gates and eliminates the need for 2's complement operations, in contrast to conventional adders and subtractors.
- Operation is governed by select-line activity, resulting in higher speed and reduced power consumption.
- Significant area savings are achieved due to minimized routing resources, avoiding extensive interconnections required for carry and borrow propagation in FPGA implementations.

Summary

- Proposed RMFAS architecture reduced the garbage values (66.66%), number of reversible logic gates (62.5%), quantum cost (45%) better than the existing RIAS architecture.
- The RMFAS architecture has the delay of 0.908 ns and occupies the area of about 1%.
- The proposed RMFAS CORDIC II rotator has improved the area occupancy of 77.87% and mitigated the arithmetic units to 100% at the cost of the number of MUX is increased to 51.6% with over VIRTEX7 FPGA devices.
- The use of reversible logic gates over CORDIC II reduced the power consumption 100% (Theoretically)and extended the life of the gadget.

A Novel One Step Trivial Rotator Based on Reversible Logic Gate

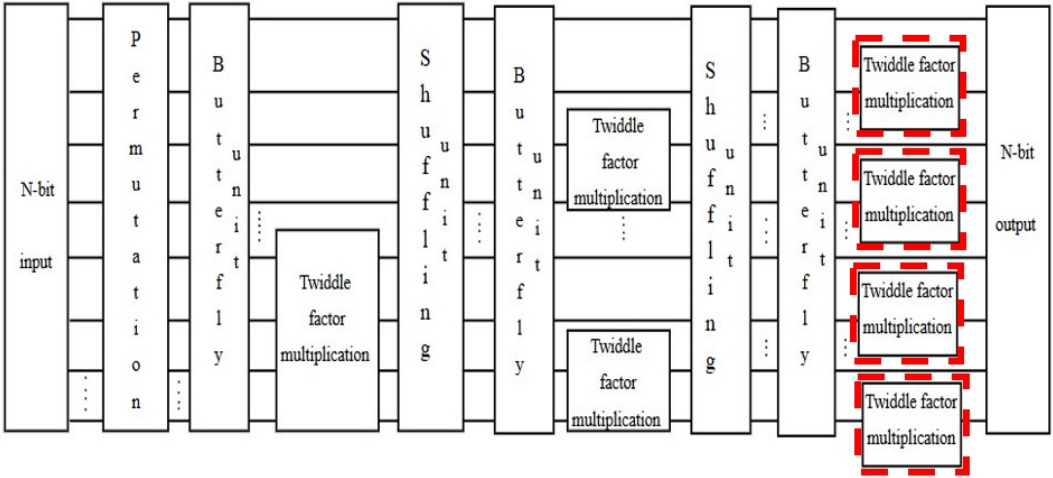


Figure. 37: Block Diagram of DIT FFT

Introduction

Trivial rotator is also used in the last stage of radix-2 FFT and also in CORDIC II rotator for rotating the input vectors.

Limitations:

- Existing methods focus on reducing the hardware complexity only on arithmetic units and less attention were given to the multiplexers complexity.

Merits of proposed gate:

- In this module, a novel 4×4 reversible One-Step Trivial Rotator (OSTR) gate was proposed for optimizing the multiplexer complexity in trivial rotator.
- The bijective OSTR gate enables single-step rotation for $\theta \in \{0^\circ, 90^\circ\}$ or $\{180^\circ, 270^\circ\}$, depending on the control signal. It also supports XOR, XNOR and NOT operations.

Trivial Rotator

- The trivial rotator begins with the initialization stage, where the input data points (x, y) are rotated to the output data points (X, Y), to a desired rotation angle θ, which are set to pre-computed micro rotation angles.

- The equation of twiddle factor is given by Eq. 23, which can be generalized for both clockwise and anti-clockwise rotation with the change in their sign depending on the direction of rotation.

$$W_N^k = \begin{cases} e^{\frac{-2\pi ik}{N}} & \text{for clockwise rotation} \\ e^{\frac{2\pi ik}{N}} & \text{for counter-clockwise rotation} \end{cases} \tag{23}$$

where $k = 0, 1, \ldots, N - 1$

Continued...

- The angle of rotation for trivial rotator is given by, θ_k as in Eq. 24

$$\theta_k = \frac{2\pi k}{N} \tag{24}$$

- Consider k=0, 1, 2 and 3, as the trivial rotator rotates the input vectors to four predefined angles in the polar co-ordinates.

- At k=0, the angle of rotation is 0^0 and the corresponding trivial factor coefficient is 1, then, at complex plane the $Z^{'} = x + iy$, given in Eq. 25 and Eq. 26.

$$\theta_k = 2\pi\frac{0}{4} = 0^\circ \tag{25}$$

$$W_N^0 = e^{i(0)} = e^0 = 1 \tag{26}$$

Continued...

Similarly, considering the other values of K, the angle obtained is presented in Table. 15.

Table 15: Tabulated Relationship of Angle, Coefficient, Switch Control, and Output

Angle	**Coefficient**	S_1	S_2	S_3	S_4	**Output**
0°	1	1	1	1	1	p, q
90°	j	1	0	0	1	$p, -q$
180°	-1	0	1	1	1	$-p, -q$
270°	$-j$	0	0	1	1	$-p, q$

Architecture of existing trivial rotator

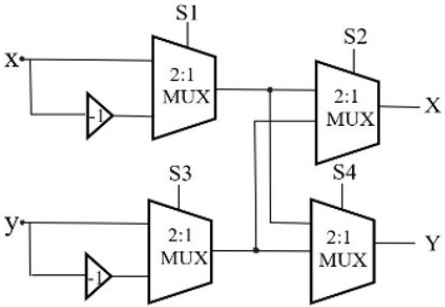


Figure. 38: Trivial rotator

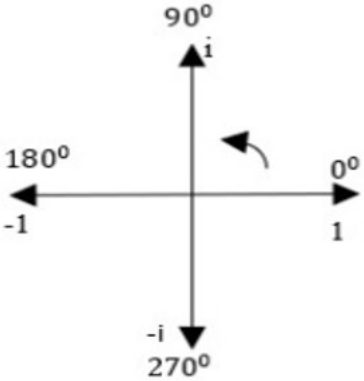


Figure. 39: Angle mapping in complex plane

Proposed OSTR gate

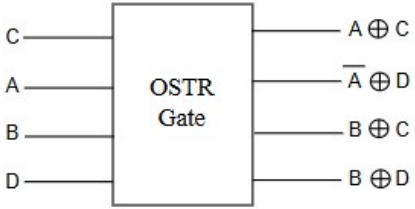


Figure. 40: 4x4 OSTR gate

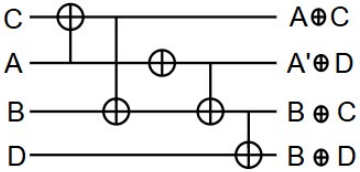


Figure. 41: Quantum realization

Truth Table of the Proposed OSTR Gate

Table 16: Truth table of the proposed OSTR gate

A	B	C	D	$P = A \oplus C$	$Q = B \oplus C$	$R = A' \oplus D$	$S = B \oplus D$
0	0	0	0	0	0	1	0
0	0	1	0	0	1	1	1
0	1	0	0	1	0	0	0
0	1	1	0	1	1	0	1
0	0	0	1	0	0	0	1
0	0	1	1	0	1	0	0
0	1	0	1	1	0	1	1
0	1	1	1	1	1	1	0
1	0	0	0	1	1	1	0
1	0	1	0	1	0	1	1
1	1	0	0	0	1	0	0
1	1	1	0	0	0	0	1
1	0	0	1	1	1	0	1
1	0	1	1	1	0	0	0
1	1	0	1	0	1	1	1
1	1	1	1	0	0	1	0

OSTR Gate Truth Tables

Table 17: Truth table for the OSTR gate as orthogonal trivial rotator

Input		**Control Signal**		**Output**				θ	**Coeff.**	**Result**
x	y	0	0	x	y	$\overline{x}$	y	0°	1	(x, y)
x	y	0	0	x	y	$\overline{x}$	y	90°	i	$(-y, x)$
x	y	1	1	$\overline{x}$	$\overline{y}$	x	y	180°	-1	$(-x, -y)$
x	y	1	1	$\overline{x}$	$\overline{y}$	x	y	270°	$-i$	$(y, -x)$

Table 18: Truth table for the OSTR gate as anti-parallel trivial rotator

Input		**Control Signal**		**Output**				θ	**Coeff.**	**Result**
x	y	0	1	x	y	$\overline{x}$	y	0°	1	(x, y)
x	y	0	1	x	y	$\overline{x}$	y	270°	$-i$	$(x, -y)$
x	y	1	0	$\overline{x}$	$\overline{y}$	x	y	180°	-1	$(-x, -y)$
x	y	1	0	$\overline{x}$	$\overline{y}$	x	y	90°	i	$(-y, x)$

Results and Discussion

Table 19: Comparison of Existing and Proposed Trivial Rotation Control Configurations

Input	**Existing [28]**						**Proposed**			
	S_1	S_2	S_3	S_4	**Angle**	**Result**	C_1	C_2	**Angle**	**Result**
(x, y)	1	1	1	1	0°	(x, y)	0	0	0°, 90°	(x, y), $(-x, y)$
(x, y)	1	0	0	1	90°	$(-x, y)$	1	1	180°, 270°	$(-x, -y)$, $(x, -y)$
(x, y)	0	1	1	1	180°	$(-x, -y)$	0	1	0°, 270°	(x, y), $(x, -y)$
(x, y)	0	0	1	1	270°	$(x, -y)$	1	0	180°, 90°	$(-x, -y)$, $(-x, y)$

Continued...

Table 20: Metrics comparison of existing and proposed trivial rotator across all angles

Metrics	**Existing ([28])**	**Proposed**
Primitive logic gates count	88	42
Input control signal count	16	4
Power consumption (mW)	0.17	0.12
LUT	8	4
Delay (ns)	0.865	0.670
Quantum Cost	Not applicable	8

Summary

- The bijective 4x4 OSTR gate enables single-step rotation for $\theta \in \{0°, 90°\}$ or $\{180°, 270°\}$, depending on the control signal.
- The OSTR gate can perform various other Boolean functions, such as XOR, XNOR, NOT, and copy.
- OSTR gate has the quantum cost of 4 and produces zero garbage value when it operates as rotator.
- The OSTR gate achieves significant reductions in hardware cost (57.5%), LUTs (50%), power (29.41%), and delay (22.54%) on an FPGA Virtex7.

Design of efficient parallel memoryless shuffling/MDC circuit

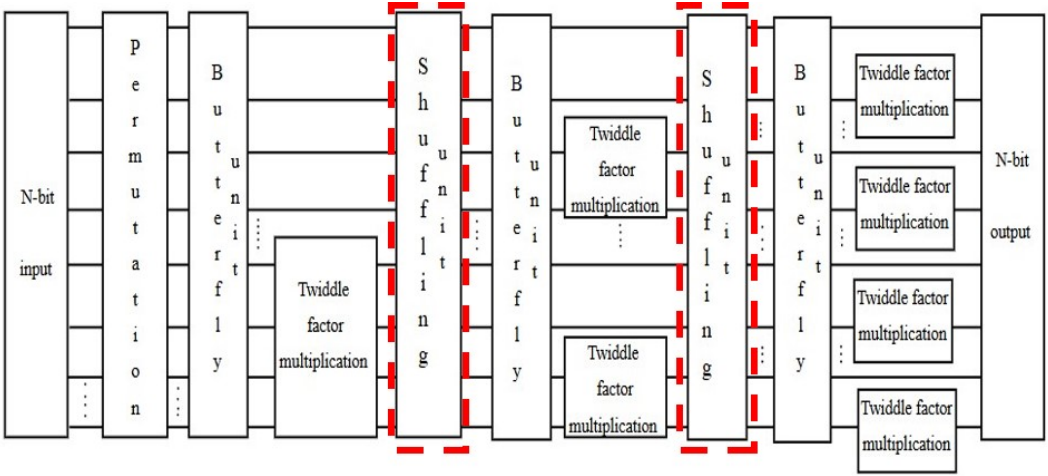


Figure. 42: Block Diagram of DIT FFT

Introduction

- A shuffling circuit is a hardware or logical arrangement that reorders data based on a specific pattern.
- In the context of Radix-2 FFT, the shuffling circuit maps the output indices of the previous intermediate stage to the next butterfly stages in a specific shuffled order.

Limitations in existing gates:

- It uses an in-place strategy realized by interchanging storage locations of butterfly outputs, which can completely remove the bank conflicts.
- It uses dual memory banks, and the number of multiplexers (MUXs) used is higher, leading to more occupancy, power, and delay.

Merits of proposed gate:

- This method uses no memory in the shuffling circuit with continuous data flow. It stores and delays data using only D flip-flops until processing is complete.
- Multiplexer banks are replaced by barrel shift registers, which deliver the required data to the correct output in a single clock cycle.

Types of Shuffling Circuit

In-Place memory reuses the same storage locations for FFT computation, making it area-efficient but requiring complex address control and offering lower throughput.

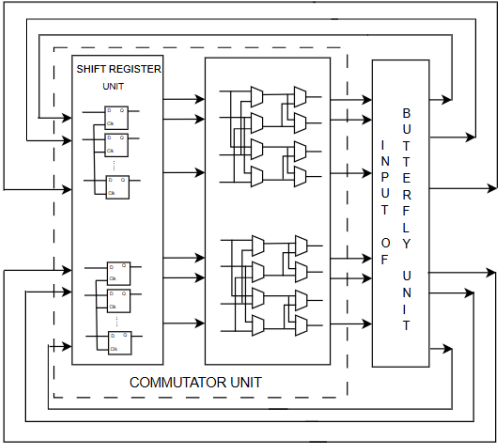


Figure. 43: General Architecture of In-Place memory

Continued...

Continuous-Flow memory uses dual/multi-bank buffers to allow data to stream through without overwriting, enabling pipelined processing and higher throughput at the cost of extra memory and control logic.

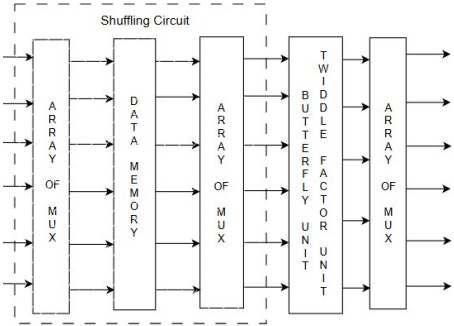


Figure. 44: General Architecture of Continuous flow memory

Proposed Shuffle/MDC circuit

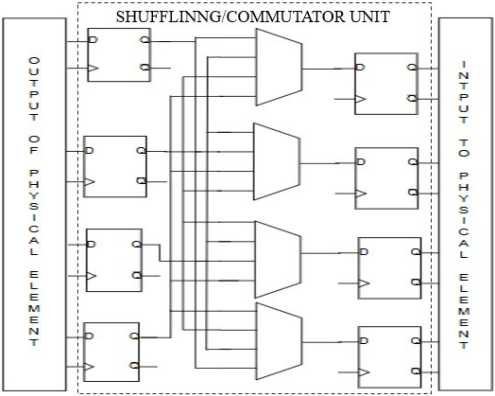


Figure. 45: Illustration of Barrel shift register based shuffling circuit

Integration of Radix-2 MDC FFT using Reversible Logic Gates

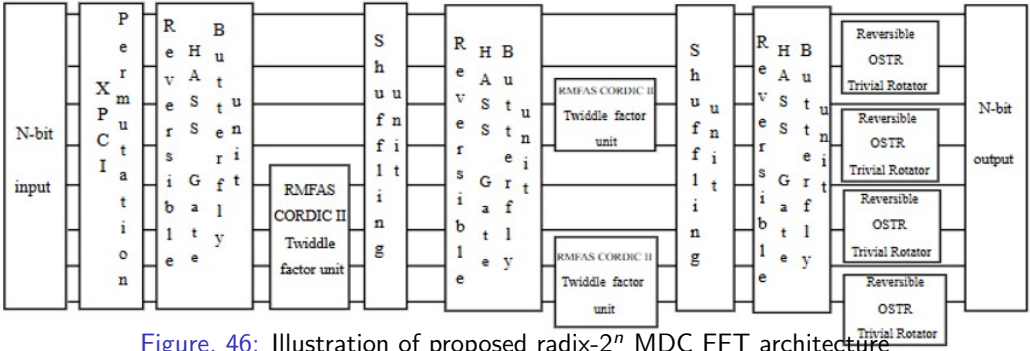


Figure. 46: Illustration of proposed radix-2^n MDC FFT architecture

- The proposed XPCI bit reversal method achieves 72.58% fewer gate count, 11.82% lower power, and 60.55% higher frequency compared to [14].
- The proposed RMFAS CORDIC II rotator reduces area by 77.87% and arithmetic units by 100%, at the cost of MUX usage by 51.6%, compared to [26].
- The proposed OSTR gate improves the speed by 22.54%, a 50% improvement in LUT utilization, and a 29.41% decrease in power consumption compared to [28].

Results and Discussions

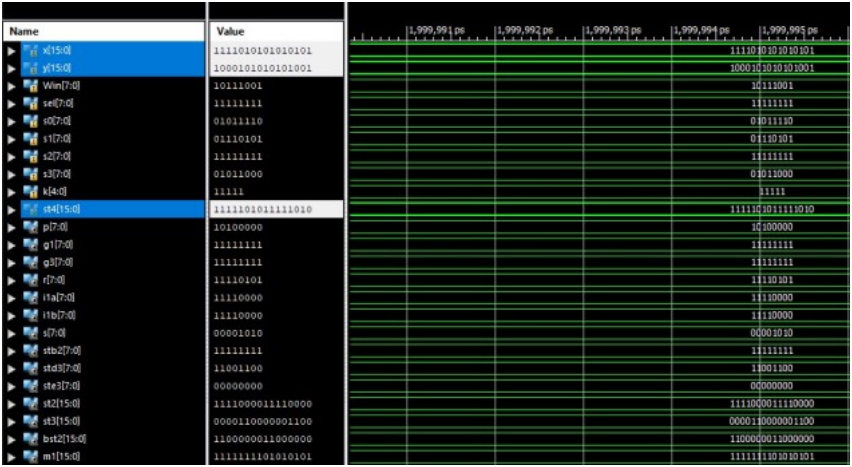


Figure. 47: Simulation result of proposed 16-bit radix-2^n MDC FFT architecture

Continued...

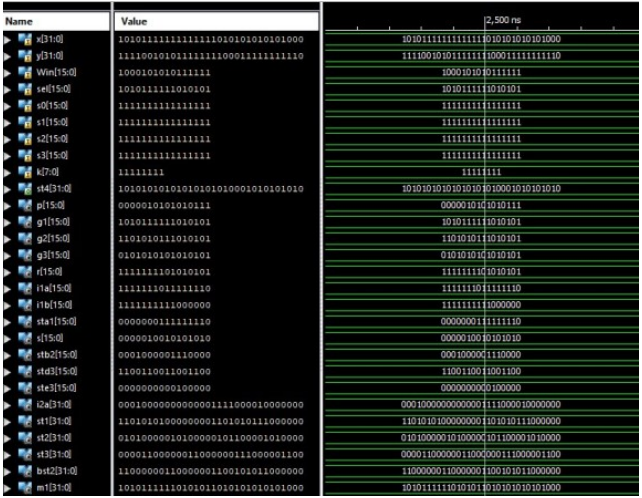


Figure. 48: Simulation result of proposed 32- bit radix-2^n MDC FFT architecture

Continued...

Table 21: Comparison of the improvement percentage for 16-point and 32-point FFT designs on FPGA

Design	**16-point FFT**			**32-point FFT**		
	LUT (%)	Power (%)	Delay (%)	LUT (%)	Power (%)	Delay (%)
J. Chen et al. [30]	263.3	126	17.607	12929	308	24.671
S. Liu et al. [38]	17.54	23.81	51.12	2.62	90.90	24.97
X. Han et al. [37]	50.46	23.07	44.59	58.155	74.14	38.95
M. Garrido et al. [39]	47.61	31.66	79.16	15.18	76.97	55.64
S. Wang et al. [7]	35.66	25.31	57.87	18.51	45.35	40.624
Proposed (Memory MDC FFT)	**91.2**	**93.5**	**10.6**	**92.6**	**88.84**	**9.1**
Proposed (Pipeline MDC FFT)	**91.89**	**80.50**	**45.47**	**93.22**	**70.74**	**41.79**

Summary

- Proposed a compact FFT design using RMFAS-CORDIC II and HASS reversible logic gate for accurate, low-power calculations, while reducing hardware size, cost, and garbage outputs.
- The design was tested on a Xilinx Virtex-7 FPGA for 16-point and 32-point FFTs. For the 16-point FFT, it reduces power use by up to 93.5%, area by 91.2%, and latency by 10.6 % compared to traditional design [7]. Similar improvements were seen for the 32-point FFT. (Using the perfect shuffle circuit)
- The architecture was implemented on a Xilinx Virtex-7 FPGA and tested with 16-point and 32-point FFTs. For the 16-point FFT, it reduces power consumption by 80.50%, area by 91.89%, and latency by 45.47% compared to traditional design [7]. Similar improvements were also seen with the 32-point FFT. (Using the proposed shuffling circuit)
- The proposed architecture demonstrated significant improvements in LUT usage, power consumption, and latency.

Conclusion

In conclusion, this work presents a compact and power-efficient FFT architecture that combines several novel design strategies.

- The use of XPCI-based index reversal enables single-cycle index permutation, while the RMFAS-CORDIC II ensures efficient twiddle factor generation.
- The use of XPCI-based index reversal enables single-cycle index permutation, while the RMFAS-CORDIC II ensures efficient twiddle factor generation.
- The introduction of the novel reversible logic gates as: HASS gate for butterfly operations, and the OSTR gate for trivial rotations further enhances computational efficiency.
- Additionally, the barrel shift register based commutator circuit supports effective MDC reordering at single clock.

Overall, the proposed architecture significantly reduces arithmetic complexity, logic depth, and hardware overhead, and its effectiveness has been validated for both 16-point and 32-point FFT sizes. This is achieved at the cost of 2762 garbage outputs for 16 bit input sequences and 5542 garbage outputs.

Future Work

- The proposed reversible logic based architecture is adaptable to both classical and quantum computing environments.
- Future efforts will focus on implementing the design in quantum platforms and assessing its scalability, efficiency, and potential for emerging quantum applications.

Publications

International Journals

[1] A. Anjana, R. Nakkeeran, and R. Ramya Jothikumar, "Design and Analysis of RMFAS CORDIC II Rotator for FPGA," IETE Journal of Research, pp. 1–11, Apr. 2025, doi: 10.1080/03772063.2025.2486582. (Indexed in: SCIE, SCOPUS, Web of Science)

International Conferences

[1] A. Anjana and R. Nakkeeran, "Unified Reversible logic gate for Half Add/Subtract and Swap Operations," 2023 Second International Conference on Advances in Computational Intelligence and Communication (ICACIC), Puducherry, India, 2023, pp. 1-4, doi: 10.1109/ICACIC59454.2023.10435331.

[2] A. Anjana and R. Nakkeeran, "Design of Area and Power Efficient Radix-2 Butterfly Unit for FFT," IEEE Sponsored 2025 International Conference on Smart & Sustainable Technology (INCSST), 27-29 June 2025, Bangalore, India, IEEE Bangalore Section.

References I

[1] M. Garrido, "**A Survey on Pipelined FFT Hardware Architectures**," *Journal of Signal Processing Systems*, vol. 94, pp. 1345–1364, 2022.

[2] Yupu Zhao, Hong Lv, Jun Li and Lulu Zhu, "**High performance and resource efficient FFT processor based on CORDIC algorithm**", *EURASIP Journal on Advances in Signal Processing*, Springer, 2022.

[3] Z. Kaya, M. Garrido, and J. Takala, "**Memory-Based FFT Architecture With Optimized Number of Multiplexers and Memory Usage**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 8, pp. 2910–2914, Aug. 2023.

[4] M. Sun, "**An efficient XOR design based on NNI and five-input majority voter in quantum-dot cellular automata**," *Optical and Quantum Electronics*, vol. 56, 2024. DOI: 10.1007/s11082-023-05729-1.

[5] A. Khan and R. K. Shaw, "**Multilayered XOR Gate: A Quantum dot Cellular Automata (QCA) Approach**," *Journal of The Institution of Engineers (India): Series B*, 2024.

References II

[6] V. Vijay et al., "**Implementation of Fundamental Modules Using Quantum Dot Cellular Automata**," *Journal of VLSI Circuits and Systems*, vol. 4, pp. 12–19, 2022.

[7] S. Wang and J. Chen, "**A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, p. 832, 2024.

[8] H. Masip-Ardevol, J. Baylina-Melé, D. Lubarov, and J. L. Muñoz-Tapia, "**RapidUp: Multi-Domain Permutation Protocol for Lookup Tables**," *Cryptology ePrint Archive*, Paper 1050, 2022.

[9] M. Garrido, J. Grajal, M. Sanchez, and O. Gustafsson, "**Pipelined Radix-2^k Feedforward FFT Architectures**," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 1, pp. 23–32, Jan. 2013.

[10] K.-J. Yang, S.-H. Tsai, and G. C. Chuang, "**MDC FFT/IFFT Processor with Variable Length for MIMO-OFDM Systems**," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 4, pp. 720–731, Apr. 2013.

References III

[11] S. G. Chen, S. J. Huang, M. Garrido, and S.-J. Jou, "**Continuous-Flow Parallel Bit-Reversal Circuit for MDF and MDC FFT Architectures**," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 10, pp. 2869–2877, Oct. 2014.

[12] W. Li, F. Yu, and Z. Ma, "**Efficient Circuit for Parallel Bit Reversal**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 63, no. 4, pp. 381–385, Apr. 2016.

[13] M. Garrido, "**Multiplexer and Memory-Efficient Circuits for Parallel Bit Reversal**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 66, no. 4, pp. 657–661, Apr. 2019.

[14] Gyanendra, B. Raman, and B. K. Kaushik, "**Novel Bit-Reordering Circuit for Continuous-Flow Parallel FFT Architectures**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 12, pp. 3

[15] A. Kircanski, "**Analysis of Boomerang Differential Trails via a SAT-Based Constraint Solver URSA**," *IACR Cryptology ePrint Archive*, vol. 2014, no. 563, 2014.

References IV

[16] M. N. Sarfaraj and S. Mukhopadhyay, "**Realization of Tristate Pauli-X, Y and Z Gates on Photonic Crystal Using Wavelength Encoding**," *Optics Communications*, vol. 550, pp. 1–18, 2024.

[17] S. Dwivedi, S. Dubey, and K. Sharma, "**Development and Evaluation of QCA-Based Feynman and Double Feynman Gates: A Design and Analytical Study**," in *Proc. 2023 Int. Conf.*, 2023.

[18] K. Elango and K. Muniandi, "**A Novel Digital Logic for Bit Reversal and Address Generations in FFT Computations**," *Wireless Personal Communications*, vol. 128, pp. 1827–1838, 2023.

[19] Theresal, T., Sathish, K., Aswinkumar, R., "**A new design of optical reversible adder and subtractor using MZI**," Published at *International Journal of Scientific and Research Publications (IJSRP)*, vol. 5, no. 4, 2015.

References V

[20] S. Moghimi and M. R. Reshadinezhad, "**A Novel 4×4 Universal Reversible Gate as a Cost Efficient Full Adder/Subtractor in Terms of Reversible and Quantum Metrics**," *International Journal of Modern Education and Computer Science (IJMECS)*, vol. 7, no. 11, pp. 28–34, 2015.

[21] R. Sarma and R. Jain, "**Quantum Gate Implementation of a Novel Reversible Half Adder and Subtractor Circuit**," in *Proc. Int. Conf. on Intelligent Circuits and Systems (ICICS)*, pp. 72–76, 2018.

[22] M. Osman and K. El-Wazan, "**Efficient Designs of Quantum Adder/Subtractor Using Universal Reversible Gate on IBM Q**," *Symmetry*, vol. 13, no. 10, p. 1842, 2021.

[23] E. Fredkin and T. Toffoli, "**Conservative Logic**," *International Journal of Theoretical Physics*, vol. 21, pp. 219–253, 1982.

[24] M. Handique and R. Singha, "**A Modified Transformation-Template Based Synthesis Using FREDKIN/SWAP Gates in Reversible Circuits**," *Procedia Computer Science*, vol. 125, pp. 801–809, 2018.

References VI

[25] S. Bhukya and S. C. Inguva, "**Design and Implementation of CORDIC Algorithm Using Integrated Adder and Subtractor**," in *Proc. 6th IEEE Int. Conf. for Convergence in Technology (I2CT)*, pp. 1–5, 2021.

[26] S. Inguva and Seventiline, "**Implementation of FPGA Design of FFT Architecture Based on CORDIC Algorithm**," *International Journal of Electronics*, vol. 108, no. 11, pp. 1914–1939, 2021.

[27] S. Chandra Inguva and J. B. Seventiline, "**LHCORDIC: Low Power FPGA Based Implementation of CORDIC Architecture**," *International Journal of Intelligent Engineering and Systems*, vol. 12, no. 2, pp. 305–314, 2019.

[28] H. Zhang, M. Chen, S. Wan, Z. Jiao, S. Hu, and P. Song, "**An Arbitrary-Phase-Varying Digital Chirp Signal Generator Based on Improved CORDIC Algorithm and Trajectory Approximator**," *IEEE Access*, vol. 12, pp. 135362–135371, 2024.

[29] R. Andersson and M. Garrido, "**Using Rotator Transformations to Simplify FFT Hardware Architectures**," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4784–4793, Dec. 2020.

References VII

[30] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, "**Hardware Efficient Mixed Radix-25/16/9 FFT for LTE Systems**," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015.

[31] S. F. Naz and A. P. Shah, "**Reversible Gates: A Paradigm Shift in Computing**," *IEEE Open Journal of Circuits and Systems*, vol. 4, pp. 241–257, 2023.

[32] M. M. Abutaleb, "**Robust and Efficient QCA Cell-Based Nanostructures of Elementary Reversible Logic Gates**," *Journal of Supercomputing*, vol. 74, no. 11, pp. 6258–6274, Nov. 2018.

[33] A. Khan and R. K. Shaw, "**Multilayered XOR Gate: A Quantum Dot Cellular Automata (QCA) Approach**," *Journal of the Institution of Engineers (India) Series B*, 2024.

[34] S. Wang and J. Chen, "**A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation**," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 2, pp. 832–836, Feb. 2024.

References VIII

[35] A. Anjana and R. Nakkeeran, "**Unified Reversible Logic Gate for Half Add/Subtract and Swap Operations**," in *Proc. 2023 Second Int. Conf. on Advances in Computational Intelligence and Communication (ICACIC)*, Puducherry, India, pp. 1–4, 2023.

[36] A. Anjana, R. Nakkeeran, and R. Jothikumar, "**Design and Analysis of RMFAS CORDIC II Rotator for FPGA**," *IETE Journal of Research*, pp. 1–11, 2025.

[37] X. Han, J. Chen, B. Qin, and S. Rahardja, "**A Novel Area-Power Efficient Design for Approximated Small-Point FFT Architecture**," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 4816–4827, Dec. 2020.

[38] S. Liu and D. Liu, "**A High-Flexible Low-Latency Memory-Based FFT Processor for 4G, WLAN, and Future 5G**," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 3, pp. 511–523, Mar. 2019.

[39] M. Garrido and P. Malagón, "**The Constant Multiplier FFT**," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 1, pp. 322–335, Jan. 2021.

References IX

[40] J. W. Cooley and J. W. Tukey, "**An algorithm for the machine calculation of complex Fourier series**," *Math. Comput.*, vol. 19, pp. 297–301, Apr. 1965.

[41] W. Hong et al., "**Multibeam antenna technologies for 5G wireless communications**," *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017.

[42] P. K. Daram and C.-I. H. Chen, "**Smart FFT measurement for reconfigurable sensor using a wideband digital receiver**," in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203.

[43] S. Liu and D. Liu, "**A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G**," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.

[44] B. Liu et al., "**Precision adaptive MFCC based on R2SDF-FFT and approximate computing for low-power speech keywords recognition**," *IEEE Circuits Syst. Mag.*, vol. 21, no. 4, pp. 24–39, 4th Quart., 2021.

References X

[45] H.-Y. Chin, P.-Y. Tsai, and S.-Y. Lee, "**Implementation of a two dimensional FFT/IFFT processor for real-time high-resolution synthetic aperture radar imaging**," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 211–216.

[46] S.-C. Hsu, S.-J. Huang, S.-G. Chen, S.-C. Lin, and M. Garrido, "**A 128-point multi-path SC FFT architecture**," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.

[47] M. Garrido and P. Paz, "**Optimum MDC FFT hardware architectures in terms of delays and multiplexers**," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 3, Mar. 2021, pp. 1003–1007.

[48] X. Han, J. Chen, and S. Rahardja, "**A new twiddle factor merging method for low complexity and high speed FFT architecture**," in *Proc. IEEE Int. Circuits Syst. Symp.*, Sep. 2019, pp. 1–5.

[49] Song-Nien, T., J. Fu-Chiang, et al, "**Multimode Memory-Based FFT Processor for Wireless Display FD-OCT Medical Systems**," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol.61, no.12, 2014, pp. 3394-3406.

THANK
YOU